

JAVA PROGRAMMING MASTERCLASS

集合框架

「用正確的容器，裝下所需的每一筆資料」

[← 返回目錄](#)

Outline

- 第一部分：集合框架概覽
 - 什麼是集合框架、介面層次
 - Collection 介面常用方法
- 第二部分：**List** 介面
 - ArrayList、LinkedList 特性與常用方法
- 第三部分：**Set** 介面
 - HashSet、LinkedHashSet、TreeSet 比較
- 第四部分：**Map** 介面
 - HashMap、LinkedHashMap、TreeMap 比較與遍歷
- 選用指南與 **Collections** 工具類別
- 實作練習

第一部分

集合框架概覽

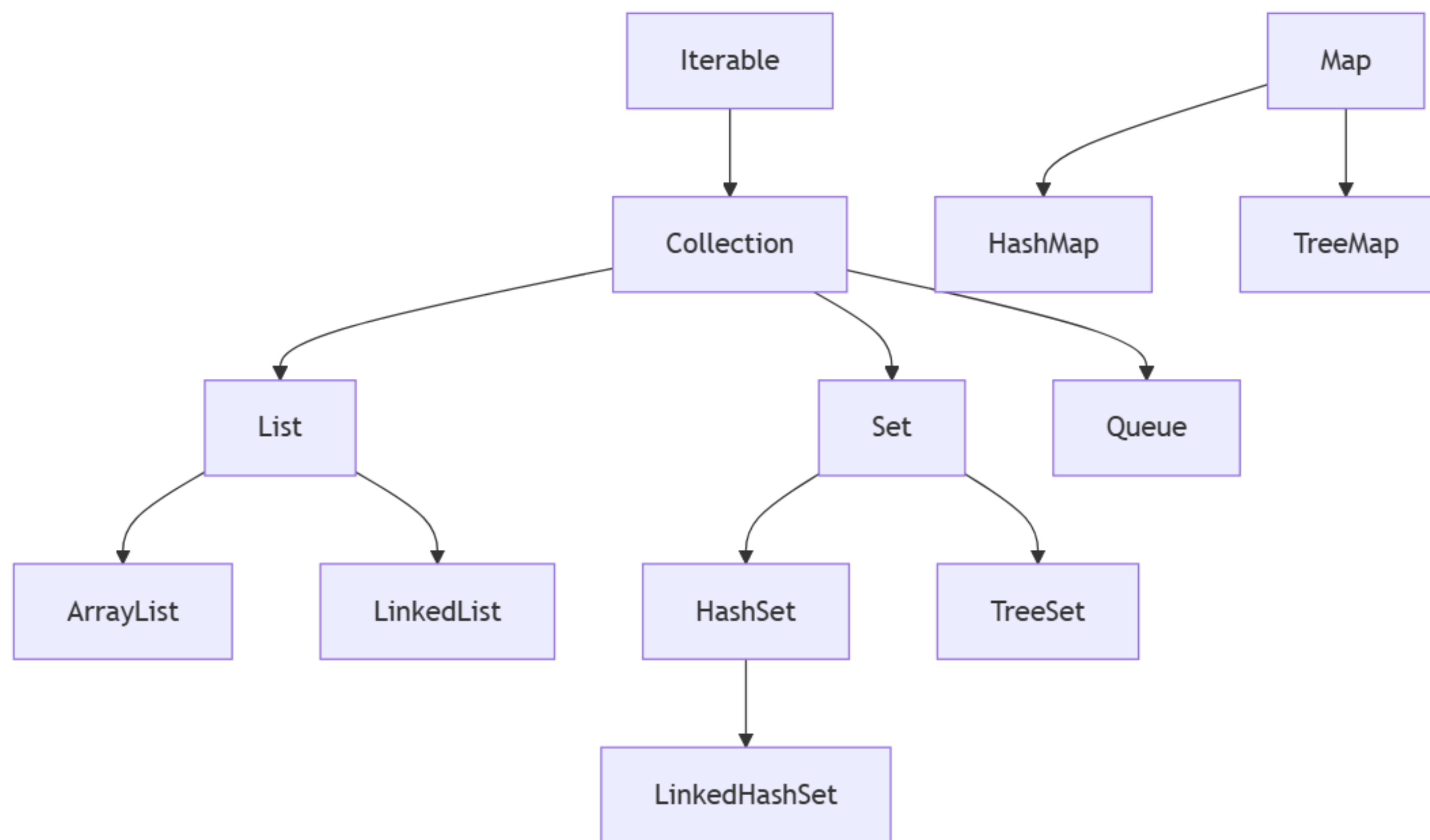
什麼是集合框架？

集合框架 (Collection Framework) 是 Java 提供的一組**標準資料結構**，讓你不必手刻就能儲存、管理與操作一群物件。

- **自動調整大小** — 無需像陣列一樣預先指定長度
- **豐富的操作方法** — 新增、刪除、搜尋、排序一應俱全
- **多種結構選擇** — 依需求選擇 List、Set 或 Map

💡 **使用時機：** 當元素數量不確定、需要快速搜尋、或需要鍵值對應時，集合框架比原始陣列更有效率。

集合框架的介面層次



Map

不繼承

Collection

，是獨立的介面體系，但同屬集合框架

Collection 介面常用方法

方法名稱

說明

add(E e)

加入一個元素

remove(Object o)

移除指定元素

contains(Object o)

是否包含該元素

size()

回傳元素數量

isEmpty()

是否為空集合

clear()

清空所有元素

iterator()

取得迭代器，用於逐一遍歷

Collection 介面方法 – 範例

```
1  import java.util.*;
2
3  List<String> fruits = new ArrayList<>();
4  fruits.add("蘋果");
5  fruits.add("橘子");
6  fruits.add("香蕉");
7
8  System.out.println(fruits.size());           // 3
9  System.out.println(fruits.contains("橘子")); // true
10 fruits.remove("橘子");
11 System.out.println(fruits.size());           // 2
12 fruits.clear();
13 System.out.println(fruits.isEmpty());        // true
```


Iterator – 迭代器遍歷

iterator() 回傳一個 **Iterator** 物件，可逐一取出元素，並在遍歷中安全地移除：

方法	說明
hasNext()	是否還有下一個元素
next()	取出下一個元素並前進
remove()	移除 next() 最後回傳的元素

```
1 List<String> fruits = new ArrayList<>(List.of("蘋果", "橘子", "香蕉"));
2 Iterator<String> it = fruits.iterator();
3 while (it.hasNext()) {
4     System.out.println(it.next());
5 }
```


不可變集合工廠方法 (Immutable Collections)

Java 9 引入了更簡潔的方式來建立不可變 (Immutable) 的集合。

- 這些集合一旦建立，不能新增、修改或刪除元素 (會拋出 `UnsupportedOperationException`)。
- 不允許 `null` 元素 (會拋出 `NullPointerException`)。

方法名稱	說明
<code>List.of(E ... elements)</code>	建立不可變的 List
<code>Set.of(E ... elements)</code>	建立不可變的 Set
<code>Map.of(K k1, V v1, ...)</code>	建立不可變的 Map (最多 10 組鍵值)

```
1 List<String> fruits = List.of("蘋果", "橘子", "香蕉");
2 Set<Integer> numbers = Set.of(1, 2, 3);
3 Map<String, Integer> scores = Map.of("炭治郎", 95, "善逸", 70);
4
5 // fruits.add("葡萄"); // 執行期會拋出例外！
```

複製為不可變集合

Java 10 新增了 `copyOf()` 方法，可以將現有的集合複製成不可變集合。

- 如果來源已經是不可變集合 (例如用 `List.of` 建立)，則直接回傳原物件，不會浪費記憶體。

方法名稱	說明
<code>List.copyOf(Collection)</code>	將集合複製為不可變 List
<code>Set.copyOf(Collection)</code>	將集合複製為不可變 Set
<code>Map.copyOf(Map)</code>	將 Map 複製為不可變 Map

```
1 List<String> mutableList = new ArrayList<>();
2 mutableList.add("A");
3 mutableList.add("B");
4
5 // 複製一份不可變的 List
6 List<String> immutableList = List.copyOf(mutableList);
7 // immutableList.add("C"); // 拋出 UnsupportedOperationException
```


第二部分

List 介面

List 介面特性

List 是有序且允許重複元素的集合，支援透過索引 (index) 存取。

```
1 List<String> list = new ArrayList<>();
2 list.add("A");
3 list.add("B");
4 list.add("A"); // 允許重複
5
6 System.out.println(list.get(0)); // "A"
7 System.out.println(list.size()); // 3
```

💡 索引從 **0** 開始，和陣列相同。宣告時通常用介面型態 **List**，實作類別用 **new ArrayList<>()**。

List 常用方法

方法名稱

說明

`get(int index)`

取得指定索引的元素

`set(int index, E e)`

替換指定位置的元素，回傳舊元素

`add(int index, E e)`

在指定位置插入元素

`remove(int index)`

移除指定位置的元素

`indexOf(Object o)`

首次出現的索引 (找不到回傳 -1)

`subList(int from, int to)`

取得子串列，範圍 [from, to-1]

List 常用方法 – 範例

```
1  List<String> heroes = new ArrayList<>();
2  heroes.add("炭治郎");
3  heroes.add("禰豆子");
4  heroes.add("善逸");
5
6  heroes.set(1, "伊之助");           // 替換索引 1
7  heroes.add(0, "煉獄");             // 在開頭插入
8  System.out.println(heroes.get(0)); // "煉獄"
9  System.out.println(heroes.indexOf("善逸")); // 3
10 List<String> sub = heroes.subList(0, 2);
11 System.out.println(sub);           // [煉獄, 炭治郎]
```


ArrayList vs LinkedList

特性	ArrayList	LinkedList
底層結構	動態陣列	雙向鏈結串列
隨機存取 <code>get(i)</code>	快 $O(1)$	慢 $O(n)$
中間插入 / 刪除	慢 $O(n)$	快 $O(1)$
記憶體用量	較少	較多 (需儲存前後節點指標)
適用場景	多讀取、少插入	多插入 / 刪除

💡 預設選 **ArrayList**，除非需要頻繁在中間插入或刪除元素，才考慮 **LinkedList**

LinkedList 的雙端操作

LinkedList 同時實作 **Deque** 介面，可當作雙端佇列使用：

方法名稱	說明
addFirst(E e)	加到串列開頭
addLast(E e)	加到串列末端
removeFirst()	移除並回傳開頭元素
removeLast()	移除並回傳末端元素

```
1  LinkedList<String> queue = new LinkedList<>();
2  queue.addLast("任務A");
3  queue.addLast("任務B");
4  System.out.println(queue.removeFirst()); // "任務A"
```


第三部分

Set 介面

Set 介面特性

Set 是不允許重複元素的集合，加入重複元素時會被自動忽略。

```
1 Set<String> names = new HashSet<>();
2 names.add("禰豆子");
3 names.add("炭治郎");
4 names.add("禰豆子"); // 重複，被忽略
5
6 System.out.println(names.size()); // 2
7 System.out.println(names.contains("炭治郎")); // true
```

💡 **Set** 沒有 **get(index)** 方法，通常用 for-each 或 Iterator 遍歷

HashSet / LinkedHashSet / TreeSet 比較

類別	順序	允許 Null	效能
HashSet	不保證順序	允許一個	最快
LinkedHashSet	維持插入順序	允許一個	中
TreeSet	依自然排序 (升序)	不允許	較慢

```
1 Set<Integer> hs = new HashSet<>(List.of(3, 1, 2));
2 Set<Integer> ts = new TreeSet<>(List.of(3, 1, 2));
3 System.out.println(hs); // [1, 2, 3] 或其他順序
4 System.out.println(ts); // [1, 2, 3] (一定升序)
```


Set 實用操作：聯集與交集

```
1  Set<String> setA = new HashSet<>(List.of("A", "B", "C"));
2  Set<String> setB = new HashSet<>(List.of("B", "C", "D"));
3
4  // 聯集：A ∪ B
5  Set<String> union = new HashSet<>(setA);
6  union.addAll(setB);
7  System.out.println(union); // [A, B, C, D]
8
9  // 交集：A ∩ B
10 Set<String> inter = new HashSet<>(setA);
11 inter.retainAll(setB);
12 System.out.println(inter); // [B, C]
```

第四部分

Map 介面

Map 介面特性

Map 儲存鍵值對 (Key-Value Pair)，每個鍵唯一，對應一個值。

```
1  Map<String, Integer> scores = new HashMap<>();
2  scores.put("炭治郎", 95);
3  scores.put("善逸", 70);
4  scores.put("炭治郎", 99); // 相同鍵 → 覆蓋舊值
5
6  System.out.println(scores.get("炭治郎")); // 99
```

💡 鍵不能重複；若用相同鍵再次 **put**，舊值會被新值覆蓋

Map 常用方法

方法名稱

說明

put(K key, V value)

存入鍵值對 (已存在則覆蓋)

get(Object key)

取得鍵對應的值 (找不到回傳 null)

getOrDefault(K key, V def)

找不到鍵時回傳預設值

remove(Object key)

移除指定鍵的鍵值對

containsKey(Object key)

是否包含指定鍵

containsValue(Object value)

是否包含指定值

size()

鍵值對數量

keySet()

取得所有鍵 (回傳 **Set**)

Map 常用方法 – 範例

```
1  Map<String, Integer> map = new HashMap<>();
2  map.put("A", 10);
3  map.put("B", 20);
4  map.put("A", 99); // 覆蓋舊值
5
6  System.out.println(map.get("A"));           // 99
7  System.out.println(map.getDefault("C", 0)); // 0
8  System.out.println(map.containsKey("B"));   // true
9  System.out.println(map.keySet());           // [A, B]
10 System.out.println(map.values());           // [99, 20]
```


HashMap vs LinkedHashMap vs TreeMap

特性	HashMap	LinkedHashMap	TreeMap
鍵的順序	不保證	維持插入順序	依鍵升序排列
允許 Null 鍵	允許一個	允許一個	不允許
存取效能	$O(1)$	$O(1)$	$O(\log n)$
適用場景	快速查詢	需維持插入順序	需依鍵排序輸出

HashMap vs LinkedHashMap vs TreeMap – 範例

```
1  Map<String, Integer> hm = new HashMap<>();
2  hm.put("banana", 2); hm.put("apple", 5);
3  System.out.println(hm); // 不保證順序
4  Map<String, Integer> lhm = new LinkedHashMap<>();
5  lhm.put("banana", 2); lhm.put("apple", 5);
6  System.out.println(lhm); // {banana=2, apple=5} (插入順序)
7  Map<String, Integer> tm = new TreeMap<>();
8  tm.put("banana", 2); tm.put("apple", 5);
9  System.out.println(tm); // {apple=5, banana=2} (鍵升序)
```


Map 的遍歷方式

```
1  Map<String, Integer> scores = new HashMap<>();
2  scores.put("炭治郎", 95);
3  scores.put("善逸", 70);
4
5  // 方式一：entrySet 遍歷 (最常用)
6  for (Map.Entry<String, Integer> entry : scores.entrySet()) {
7      System.out.println(entry.getKey() + " → " + entry.getValue());
8  }
9
10 // 方式二：keySet 遍歷
11 for (String key : scores.keySet()) {
12     System.out.println(key + " → " + scores.get(key));
13 }
```

選用指南與工具類別

如何選擇集合類別

需求

選用

有序、可重複、需快速隨機存取

ArrayList

有序、可重複、頻繁插入 / 刪除

LinkedList

無重複、不在乎順序

HashSet

無重複、需維持插入順序

LinkedHashSet

無重複、需排序

TreeSet

鍵值對應、快速查詢

HashMap

鍵值對應、需維持插入順序

LinkedHashMap

鍵值對應、需依鍵排序

TreeMap

Collections 工具類別

java.util.Collections 提供操作集合的靜態工具方法：

方法名稱	說明
sort(List)	將 List 升序排序
reverse(List)	反轉 List 的順序
shuffle(List)	隨機打亂 List
max(Collection)	取得最大值
min(Collection)	取得最小值

Collections 工具類別 – 範例

```
1  import java.util.*;
2
3  List<Integer> nums = new ArrayList<>(List.of(3, 1, 4, 1, 5));
4
5  Collections.sort(nums);
6  System.out.println(nums);           // [1, 1, 3, 4, 5]
7
8  Collections.reverse(nums);
9  System.out.println(nums);           // [5, 4, 3, 1, 1]
10
11 System.out.println(Collections.max(nums)); // 5
12 System.out.println(Collections.min(nums)); // 1
```


練習一：管理英雄名單

任務說明

請宣告一個 `ArrayList<String>`，儲存以下鬼殺隊成員：「炭治郎、禰豆子、善逸、伊之助、蜜璃」

完成以下操作：

1. 在「善逸」前面插入「甘露寺」
2. 將「禰豆子」替換為「時透無一郎」
3. 移除最後一個成員
4. 用 `Collections.sort()` 依字典順序排序後印出

練習一：解題提示

提示說明

1. 善逸在索引 2，使用 `list.add(2, "甘露寺")` 插入
2. 禰豆子在索引 1，使用 `list.set(1, "時透無一郎")` 替換
3. `list.remove(list.size() - 1)` 移除最後一個元素
4. `Collections.sort(list)` 排序

```
1 List<String> m = new ArrayList<>(
2     List.of("炭治郎", "禰豆子", "善逸", "伊之助", "蜜璃"));
3 m.add(2, "甘露寺");
4 m.set(1, "時透無一郎");
5 m.remove(m.size() - 1);
6 Collections.sort(m);
7 System.out.println(m);
```


練習二：成績統計系統

任務說明

宣告一個 `HashMap<String, Integer>` 儲存以下成績：炭治郎：95、善逸：70、伊之助：85、蜜璃：90

完成以下操作：

1. 新增「甘露寺：88」
2. 將「善逸」的成績更新為 80
3. 計算全班平均分數（整數）
4. 印出所有成績 ≥ 85 的學生姓名

練習二：解題提示

提示說明

1. `map.put("甘露寺", 88)` — 新增
2. `map.put("善逸", 80)` — 覆蓋舊值即為更新
3. 用 `values()` 取得所有分數，加總後除以 `size()`
4. 用 `entrySet()` 遍歷，判斷 `entry.getValue() ≥ 85`

```
1  int total = 0;
2  for (int s : scores.values()) total += s;
3  System.out.println("平均：" + total / scores.size());
4  for (var e : scores.entrySet())
5      if (e.getValue() ≥ 85)
6      System.out.println(e.getKey());
```

實戰加料

為期中作業暖身 🔥

實戰：用 Enum 管理問卷狀態

問卷有三種狀態，用字串容易打錯字；用 `enum` 讓編譯器幫你把關：

```
1  enum QuizStatus {
2      NOT_STARTED("尚未開始"),
3      ACTIVE("進行中"),
4      ENDED("已結束");
5
6      private final String label;
7      QuizStatus(String label) { this.label = label; }
8      public String getLabel() { return label; }
9  }
```

```
1  QuizStatus status = QuizStatus.ACTIVE;
2  System.out.println(status.getLabel()); // 進行中
3
4  // 搭配 switch 更清晰
5  switch (status) {
6      case NOT_STARTED → System.out.println("問卷連結停用");
7      case ACTIVE      → System.out.println("可以填寫問卷！");
8      case ENDED       → System.out.println("問卷已截止，可查看統計");
9  }
```

實戰：List 分頁查詢

前台列表「每頁 10 筆」的分頁邏輯，`subList()` 一行搞定：

```
1  List<String> allQuizzes = new ArrayList<>(
2      List.of("問卷A", "問卷B", "問卷C", "問卷D", "問卷E",
3              "問卷F", "問卷G", "問卷H", "問卷I", "問卷J", "問卷K"));
4
5  int pageSize = 10;
6  int page = 1; // 目前第幾頁 (從 1 開始)
7
8  int from = (page - 1) * pageSize;
9  int to = Math.min(from + pageSize, allQuizzes.size());
10
11 if (from ≥ allQuizzes.size()) {
12     System.out.println("此頁無資料");
13 } else {
14     List<String> pageData = allQuizzes.subList(from, to);
15     System.out.println("第 " + page + " 頁：" + pageData);
16 }
17 // 第 1 頁：[問卷A, 問卷B, ..., 問卷J]
```

💡 關鍵：`Math.min()` 防止最後一頁筆數不足時發生 `IndexOutOfBoundsException`。

實戰：Map 統計選項票數

問卷統計頁要顯示每個選項的得票數，`Map + getOrDefault()` 是最直覺的解法：

```
1 // 模擬從資料庫撈出的所有作答（單選題）
2 List<String> answers = List.of("A", "B", "A", "C", "A", "B", "C",
"C");
3
4 // Step 1：計票
5 Map<String, Integer> tally = new HashMap<>();
6 for (String ans : answers) {
7     tally.put(ans, tally.getOrDefault(ans, 0) + 1);
8 }
9 System.out.println(tally); // {A=3, B=2, C=3}
10
11 // Step 2：依票數降序輸出（統計頁面排列用）
12 tally.entrySet().stream()
13     .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
14     .forEach(e →
15         System.out.printf("選項 %s : %d 票 (%.1f%%)\n",
16             e.getKey(), e.getValue(),
17             e.getValue() * 100.0 / answers.size()));
```


實戰：多選題的拆解與統計

多選題答案用分號 `;` 串接存入資料庫，統計時要先拆開再計票：

```
1 // 從資料庫撈出的多選答案（每筆是一個受訪者的作答）
2 List<String> rawAnswers = List.of("A;B", "B;C", "A;C", "A;B;C", "B");
3
4 Map<String, Integer> tally = new HashMap<>();
5 for (String raw : rawAnswers) {
6     String[] options = raw.split(";"); // ← 依分號拆解
7     for (String opt : options) {
8         tally.put(opt, tally.getOrDefault(opt, 0) + 1);
9     }
10 }
11 System.out.println(tally); // {A=3, B=4, C=3}
```

⚠ 注意：存入時用 `String.join(";", selectedOptions)` 串接；讀出時用 `split(";")` 還原。
記得兩端統一格式，不然 `" A;B"` 和 `"A;B"` 會被視為不同選項！

Q & A

課程結束

掌握集合框架，資料管理更有效率！

如有課後疑問，歡迎來信討論。